SMART BLIND ASSISTANT SYSTEM
(VISIONMATE APP)

Project Report submitted to the

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

for the award of the degree

MASTER OF COMPUTER APPLICATIONS

Submitted by

TIPPANI UDAY KIRAN

Reg.No: 248865100055

Under the esteemed guidance of

Dr. D. DASU

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

UNIVERSITY COLLEGE OF ENGINEERING

ADIKAVI NANNAYA UNIVERSITY, RAJAMAHENDRAVARAM, A.P.

RAJAMAHENDRAVARAM – 533296

2024-2026

ADIKAVI NANNAYA UNIVERSITY, RAJAMAHENDRAVARAM

UNIVERSITY COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Project report entitled "**SMART BLIND ASSISTANT SYSTEM (VISIONMATE APP)**" is being submitted by **TIPPANI UDAY KIRAN** Registration Number: **248865100055** to the **Department of Computer Science and Engineering, University College of Engineering, Adikavi Nannaya University,** Rajamahendravaram, Andhra Pradesh, India, is a record of bonafide project work carried out by him under my supervision and guidance and is worthy of consideration for the award of the degree of **Master of Computer Applications** .

Project Guide **Head of the Department**

External Examiner

DECLARATION

I certify that:

a. the work contained in this report is original and has been done by me under the guidance of my supervisor(s).

b. the work has not been submitted to any other Institute for any degree or diploma.

c. I have followed the guidelines provided by the university in preparing the report.

d. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the institute.

e. Whenever I have used materials (data, theoretical analysis, figures and text) from other sources, I have given due credit to them by citing them in the text of the report and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

Signature of the Student

ACKNOWLEDGEMENT

I feel fortunate to pursue my **Master of Computer Applications** degree from the campus of **Department of Computer Science and Engineering, University College of Engineering, Adikavi Nannaya University**, Rajamahendravaram. It provided all the facilities in the field of Computer Science and Engineering.

It's a genuine pleasure to express my deep sense of thanks and gratitude to my mentor and project guide **Dr. D. Dasu, Assistant Professor, Department of Computer Science and Engineering, Adikavi Nannaya University** for his excellent guidance right from selection of project and his valuable suggestions throughout the project work. His constant and timely counsel has been the cause for me to succeed in completing this in college. He has given me tremendous support in the technical front.

I profusely thank **Dr. P. Venkateswara Rao, Principal of University College of Engineering (UCE)** for all the encouragement and support.

I also thank **Dr. B. Kezia Rani, Head of the Department, Department of Computer Science and Engineering (CSE)** for her guidance throughout the academics.

I also thank **Dr. D. Latha, Project Coordinator, Department of Computer Science and Engineering (CSE)** for her guidance throughout the academics.

A great deal of thanks goes to **Review Committee Members** and the entire faculty members of the **Department of Computer Science and Engineering, University College of Engineering, Adikavi Nannaya University,** Rajamahendravaram department for their excellent supervision and valuable suggestions.

I am always thankful to my family, friends and well-wishers for all their trust and constant support in the successful completion of my project.

TIPPANI UDAY KIRAN

Reg.No: 248865100055

ABSTRACT

Visual impairment significantly affects the mobility and independence of individuals in their daily lives. Traditional assistive tools such as white canes provide limited support and are not effective in detecting dynamic obstacles in real time. This project presents Visionmate, a smart mobile-based assistance system that uses deep learning and computer vision techniques to help visually impaired users navigate their surroundings safely.

The system utilizes a real-time camera to capture environmental frames, which are processed using a YOLOv8-based object detection model**.** The model is trained using a hybrid dataset combining the RDD2022 road damage dataset and a speed bump dataset**,** along with transfer learning to improve detection performance. It is capable of detecting obstacles such as pedestrians, vehicles, potholes, and speed bumps.

Based on the detected objects and their positions, the system provides real-time audio feedback using text-to-speech technology. In addition to obstacle detection, the application also integrates an AI voice assistant that allows users to perform tasks such as opening mobile applications, sending WhatsApp messages to contacts, making phone calls, accessing contacts, opening Google Maps for navigation, and retrieving information such as current date, time, location, and battery status through voice commands.

The application operates in real time without requiring continuous internet connectivity, making it suitable for practical everyday use. This system provides a reliable and efficient solution to improve the safety, accessibility, and independence of visually impaired individuals.

KEYWORDS: ***Assistive Technology, YOLOv8, Object Detection, Computer Vision, Visual Impairment, Voice Assistant, Smart Navigation, Hybrid Dataset.***

.

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

1.1. Introduction to Project

Visual impairment affects the mobility and independence of millions of people around the world. Navigating through roads, public places, and unfamiliar environments can be difficult for visually impaired individuals due to the inability to detect obstacles such as vehicles, pedestrians, potholes, and other hazards. Traditional assistive tools such as white canes provide basic support, but they have limitations in detecting obstacles at a distance and providing detailed information about the surroundings.

With the advancement of artificial intelligence and computer vision, smart assistive systems have been developed to improve the safety and mobility of visually impaired users. These systems can analyze the surrounding environment in real time and provide useful feedback to help users navigate more safely.

This project focuses on developing Visionmate, a smart mobile-based assistance system that uses deep learning techniques for real-time obstacle detection. The system uses a YOLOv8 object detection model trained on a hybrid dataset combining the RDD2022 road damage dataset and a speed bump dataset. The model is capable of detecting obstacles such as pedestrians, vehicles, potholes, and speed bumps.

The application processes camera frames in real time and provides voice alerts using text-to-speech technology to inform users about nearby obstacles. In addition to obstacle detection, the system also includes an AI voice assistant that allows users to perform tasks such as opening applications, sending WhatsApp messages, making phone calls, accessing contacts, opening Google Maps for navigation, and retrieving information like current date, time, location, and battery status.

This system provides a practical solution to improve the safety, accessibility, and independence of visually impaired individuals by combining computer vision, deep learning, and voice assistance technologies.

1.2. Problem Statement

Visual impairment significantly affects the ability of individuals to navigate safely in their surroundings. Obstacles such as pedestrians, vehicles, potholes, and speed bumps can create serious risks for visually impaired people while walking in public places or unfamiliar environments. Traditional assistive tools such as white canes provide limited support and are not capable of detecting obstacles at a distance or providing detailed information about the environment.

This project aims to develop Visionmate, a smart assistance system that uses deep learning–based object detection to identify obstacles using a mobile device camera. By utilizing a YOLOv8 model trained on a hybrid dataset, the system detects obstacles in real time and provides voice feedback to the user. Additionally, the system integrates an AI voice assistant to perform tasks such as opening applications, making calls, sending messages, and accessing navigation services, thereby improving the safety and independence of visually impaired individuals.

1.3. Goals & Objectives

The goal of this project is to develop an intelligent and reliable assistance system for visually impaired individuals using deep learning and computer vision technologies. The system aims to help users detect obstacles in real time and navigate their surroundings more safely. By integrating object detection and voice assistance features into a mobile application, the project provides a practical and user-friendly solution to improve the mobility, safety, and independence of visually impaired individuals.

Objectives:

1. **Dataset Collection and Preparation:**
    - To collect and prepare suitable datasets, specifically the RDD2022 road damage dataset and **a speed bump dataset**, to train the object detection model for identifying road obstacles.
2. **Data Preprocessing:**
    - To preprocess the dataset through steps such as image resizing, annotation formatting, and dataset organization to improve model training and detection performance.

3. **Model Development:**
    - To implement a YOLOv8-based object detection model capable of detecting obstacles such as pedestrians, vehicles, potholes, and speed bumps.
4. **Model Training and Evaluation:**
    - To train the object detection model using transfer learning and evaluate its performance using metrics such as accuracy, precision, and recall.
5. **System Development:**
    - To develop a mobile application (visionmate**)** that integrates the trained model to detect obstacles in real time using the device camera and provide voice alerts through text-to-speech technology.
6. **Voice Assistant Integration:**
    - To integrate an AI voice assistant that allows users to perform tasks such as opening applications, sending WhatsApp messages, making phone calls, accessing contacts, opening Google Maps for navigation, and retrieving information such as date, time, location, and battery status.
7. **Testing and Validation:**
    - To test and validate the system in real-time scenarios to ensure its reliability, accuracy, and usability for visually impaired users.
8. **Practical Application:**
    - To provide an efficient and accessible assistive solution that enhances the safety, mobility, and independence of visually impaired individuals in their daily-lives.

CHAPTER 2

LITERATURE SURVEY

The literature indicates a growing use of computer vision and deep learning techniques in assistive systems for visually impaired individuals. Early research focused on sensor-based and traditional image processing methods for obstacle detection, but these approaches had limitations in accuracy, scalability, and real-time performance in complex environments.

Recent advancements in deep learning, particularly Convolutional Neural Networks (CNNs) and object detection models such as YOLO and SSD, have significantly improved the ability of systems to detect and classify objects in real time. These methods enable automatic feature extraction from images and provide higher accuracy in identifying multiple objects and environmental obstacles.

The proposed vision-based assistive system builds upon these developments by using deep learning models for real-time object detection and voice-based feedback. This enables improved environmental awareness and supports safe and independent navigation for visually impaired users.

1. K. Naveen Kumar, Sanjeevani Sharma, and ILIN Mariam Abraham (2022)

This study proposes a **Blind Assistance System Using Machine Learning** designed to support visually impaired individuals in their daily activities. The system utilizes a camera-based object detection approach combined with machine learning techniques to identify common indoor objects such as beds, chairs, televisions, and remote controls. The detected objects are communicated to the user through voice-based feedback, helping them understand their surroundings. The research mainly focuses on assisting users in indoor environments and does not explicitly address outdoor hazards such as potholes or road obstacles.

2. Dr. V. Sivakumar, Neha N., Ria Sathya, Skandana G., Yashaswini R., and R. Swathi (2022)

This research presents a system titled **Smart Assistance for Visually Impaired and Blind People**, which utilizes artificial intelligence for real-time object detection and environmental awareness. The system is implemented using TensorFlow and the SSD (Single Shot Detector)

architecture to detect surrounding objects through a camera. It provides voice alerts to inform users about detected objects and includes a distance estimation mechanism to warn users about nearby obstacles. The system aims to enhance independent mobility and daily navigation for visually impaired individuals.

3. Maeda, H., Sekimoto, Y., Seto, T., Kashiyama, T., and Omata, H. (2018)

Maeda and colleagues introduced a large-scale road damage detection system using deep neural networks. They developed the Road Damage Dataset (RDD) and used convolutional neural networks to detect road damages such as cracks and potholes automatically. Their research demonstrated the potential of deep learning techniques for large-scale road infrastructure monitoring.

4. Zhang, L., Yang, F., Zhang, Y. D., and Zhu, Y. J. (2016)

Zhang and his team proposed an automated road crack detection system using deep convolutional neural networks. Their research showed that CNN-based models can effectively identify cracks in road surfaces with higher accuracy compared to traditional image processing methods

5. Cha, Y. J., Choi, W., and Büyüköztürk, O. (2017)

This study introduced a deep learning-based vision system for detecting structural damages such as cracks and surface defects. The researchers used convolutional neural networks to analyze images and successfully detect damaged areas in infrastructure components.

6. Gopalakrishnan, K., Khaitan, S., Choudhary, A., and Agrawal, A. (2017)

The researchers applied deep convolutional neural networks for pavement crack detection. Their work demonstrated that deep learning models can effectively identify irregular crack patterns on road surfaces and outperform traditional machine learning approaches

7. Fan, R., Ai, M., Zhu, H., and Dahnoun, N. (2019)

Fan and colleagues developed a real-time pothole detection system using deep learning techniques. The system captures road images using a camera and processes them using CNN-based classification models to detect potholes accurately.

8. Mohan, A., Poobal, S., and others (2018)

This research focused on automated pothole detection using image processing and machine learning algorithms. Feature extraction techniques were applied to identify pothole regions in road images, improving detection accuracy and efficiency.

9. Anand, S., Gupta, S., and Darbari, H. (2018)

The authors developed a computer vision-based system for detecting potholes in road images. Their work emphasized the use of machine learning techniques to improve road condition monitoring and automate road inspection processes.

10. Chen, Z., Zhang, Y., and colleagues (2020)

Chen and his team proposed deep learning-based object detection models for detecting road surface anomalies. Their study highlighted the effectiveness of training deep neural networks on large road image datasets to improve detection accuracy.

11. Arya, D., Maeda, H., Ghosh, S., Toshniwal, D., and Mraz, A. (2021)

This research compared several deep learning models for road damage detection using the RDD dataset. The authors demonstrated that YOLO-based object detection models provide efficient and accurate detection of road damages such as cracks and potholes.

12. Ultralytics Research Team (2023)

The Ultralytics research team introduced **YOLOv8**, an advanced object detection architecture designed for high accuracy and real-time performance. YOLOv8 uses deep convolutional neural networks for efficient object detection and is widely used in applications such as autonomous driving, surveillance, and assistive navigation systems.

CHAPTER 3

SYSTEM ANALYSIS

3.1. Existing System

Several existing systems have been developed to assist visually impaired individuals in navigation and object recognition. These systems mainly rely on GPS-based navigation, computer vision, mobile applications, and hardware-assisted devices to provide partial support in daily activities. However, most of these systems are limited in providing complete real-time environmental awareness and hazard detection.

1.AI-Based Accessibility Applications (e.g., Microsoft Seeing AI):

Applications like Microsoft Seeing AI use artificial intelligence and computer vision techniques to identify objects, read text, and describe surroundings. While these systems are useful for object recognition and scene description, they are not specifically optimized for real-time outdoor navigation and hazard avoidance in dynamic road conditions.

2.Human Assistance-Based Systems (e.g., Be My Eyes):

Be My Eyes is an application that connects visually impaired users with sighted volunteers through live video calls. This system provides accurate assistance based on human observation; however, it depends heavily on the availability of volunteers and requires internet connectivity, making it unreliable in emergency or real-time navigation scenarios.

3.Hardware-Based Assistive Devices:

Some assistive technologies use ultrasonic sensors or wearable devices to detect nearby obstacles. These systems can provide basic obstacle detection but are often limited in range, accuracy, and environmental understanding. Additionally, such devices are expensive and not widely accessible to all users.

Limitations of Existing Systems :

- Lack of real-time road hazard detection during navigation such as potholes, speed bumps, and dynamic obstacles.
- No proper directional obstacle avoidance guidance for safe movement.
- Some systems depend heavily on continuous internet connectivity, reducing

reliability in low-network areas.

- Hardware-based assistive devices are often expensive and not affordable for all users.
- Cloud-based processing introduces latency, affecting real-time performance.
- Limited integration between navigation assistance and intelligent safety detection systems.

3.2 Proposed System

The proposed **VisionMate system** is an AI-powered assistive navigation solution designed to help visually impaired individuals by providing real-time obstacle detection, hazard identification, and intelligent navigation support. The system integrates computer vision and machine learning techniques to analyse the surrounding environment using a smartphone camera and deliver instant feedback to the user.

The process begins with capturing real-time video or image input through a mobile device camera. The system processes this input using a trained object detection model to identify obstacles such as potholes, speed bumps, vehicles, and other road hazards. The detected objects are then analysed in real-time, and appropriate voice-based alerts are generated to guide the user safely. The system is also integrated with GPS-based navigation to assist users in reaching their destination efficiently while avoiding potential hazards. Unlike traditional systems, VisionMate performs most of the processing on-device, ensuring faster response time and reduced dependency on cloud services.

The final system is deployed as a mobile-based application that can run on standard Android smartphones, making it accessible and practical for real-world usage.

Advantages:

- **High Accuracy:** The use of deep learning-based object detection (YOLO-based models) enables accurate identification of road obstacles and hazards in real time.
- **Real-Time Detection:** Provides instant detection and alerts for obstacles such as potholes, speed bumps, and moving objects, improving user safety during navigation.

- **On-Device Processing:** Most computations are performed directly on the mobile device, reducing latency and eliminating dependency on constant internet connectivity.

- **Voice-Based Assistance:** Generates real-time audio alerts to guide visually impaired users without requiring visual interaction.

- **Integrated Navigation:** Combines GPS-based navigation with obstacle detection for safer and smarter route guidance.

- **Cost-Effective and Scalable:** Designed to run on standard Android smartphones using open-source tools, making it affordable and easily deployable for large-scale use.

Feasibility Study

The feasibility study evaluates whether the proposed **VisionMate AI-powered assistive navigation system** is practical, implementable, and effective for visually impaired individuals. The study is analyzed from economic, operational, and technical perspectives.

Economic Feasibility:

The system is developed using open-source tools, frameworks, and pre-trained machine learning models, which significantly reduce development costs. It runs on standard Android smartphones, eliminating the need for expensive specialized hardware. This makes the solution affordable and accessible, especially for users in low-resource environments.

Operational Feasibility:

The system is designed to be simple and user-friendly. Users can operate the mobile application easily, which processes real-time camera input and provides voice-based alerts for navigation. The integration of GPS navigation and obstacle detection ensures smooth usability in daily life.

Technical Feasibility:

The system is implemented using technologies such as Python, OpenCV, and deep learning models like YOLO. These tools support real-time object detection and are widely used in computer vision applications. The system can run efficiently on modern smartphones with moderate processing power, making it technically feasible.

CHAPTER - 4

SYSTEM REQUIREMENTS AND SPECIFICATION

The features and behavior of a system or software application are described in a document or group of documents called a system requirement specification. It consists of a number of components that make an effort to define the intended functionality needed by the client to fulfil their various consumers.

The system requirements specification includes four categories.

- Functional Requirements
- Non-Functional Requirements
- Software Requirements
- Hardware Requirements

4.1. Functional Requirements

Functional requirements describe the key functions and capabilities that the **VisionMate system** must provide to fulfill user needs. The system is designed to assist visually impaired individuals by enabling real-time navigation support, obstacle detection, and voice-based guidance.

- **Voice Command Input:** The system should accept voice commands from the user to initiate navigation and control application features.
- **Navigation Activation:** The system should launch and integrate with GPS-based navigation (walking mode) for route guidance.
- **Camera Activation:** The system should activate the mobile camera during navigation to capture real-time environmental input.
- **Real-Time Obstacle Detection:** The system should detect road obstacles in real time using a trained object detection model.
- **Obstacle Classification:** The system should classify detected objects into categories such as potholes, speed bumps, vehicles, poles, and other road hazards.
- **Voice-Based Alerts:** The system should generate real-time voice alerts to guide the user and ensure safe navigation.

- **AI-Based Assistance:** The system should provide intelligent assistance by combining navigation data and obstacle detection for decision support.
- **User Instruction Handling:** The system should be able to execute tasks based on user voice instructions and navigation requirements.

4.2. Non-Functional Requirements

Non-functional requirements describe the system's performance, quality, and behavioral characteristics, ensuring that the **VisionMate system** is efficient, reliable, and usable in real-world conditions.

- **Accuracy:** The system should provide high accuracy in detecting and classifying road obstacles to ensure reliable assistance for visually impaired users in different environments.
- **Performance:** The system should deliver real-time or near real-time responses with low latency (within a few seconds) to ensure smooth navigation and timely hazard alerts.
- **Usability:** The application should have a simple and user-friendly interface with voice-based interaction, making it easy for visually impaired users to operate without technical knowledge.
- **Reliability:** The system should provide consistent performance across different lighting, weather, and road conditions with minimal failures or downtime.
- **Battery Efficiency:** The application should be optimized to consume minimal battery power during continuous camera and processing usage.
- **Security:** The system should ensure safe handling of user data and navigation inputs, maintaining privacy and secure processing of real-time information.
- **Offline Capability:** The system should support partial or full offline functionality for obstacle detection to ensure usability in areas with poor or no internet connectivity.
- **Lightweight Design:** The application should be optimized to run efficiently on standard Android devices without requiring high-end hardware resources.
- **Scalability:** The system should be designed to support future enhancements such as additional obstacle classes, improved models, or expanded navigation features.

4.3. Software Requirements

Software requirements specify the technologies used in the development of the **VisionMate mobile-based assistive system** to achieve the desired functionality, including real-time obstacle detection, navigation support, and voice-based assistance.

- **Operating System:** Android (Mobile Application Deployment), Windows 11 (Development Environment)
- **Frontend Framework:** React Native with TypeScript (for cross-platform mobile application development)
- **Build Platform:** Expo EAS (for cloud-based Android application building and deployment)
- **Backend:** Python (for processing and integration of AI/ML components)
- **Database:** PostgreSQL with Drizzle ORM (for storing detected hazard logs and user-related data)
- **Development Tools:** Visual Studio Code (for coding) and Android Emulator / Physical Android Device (for testing)

4.4. Hardware Requirements

Hardware requirements specify the necessary physical components and configurations needed to support the software.

- Widows with minimum 8GB RAM and 10GB of Disk space.
- Android smartphone
- Rear camera (minimum 8MP recommended)
- Minimum 2GB device storage in mobile
- Internet connectivity (Wi-Fi / Mobile data) for testing the app .

CHAPTER - 5

SYSTEM DESIGN

5.1. System Architecture

System Architecture refers to the high-level structure of a software system, including its components and their interactions. It is a conceptual model that defines the structure, behaviour, and more views of a system. An architecture description is a formal description and representation of a system, organized in a way that supports reasoning about the structure and behaviour of the system. A representation of a system, including a mapping of functionality onto hardware and software components, a mapping of the software architecture onto the hardware architecture, and human interaction with these components.

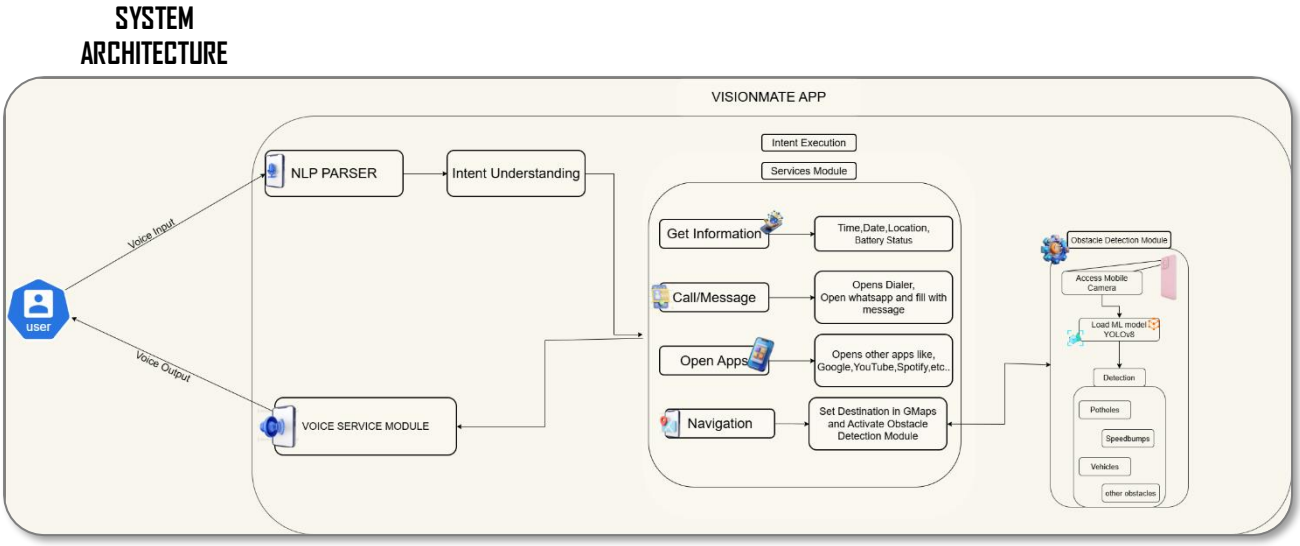


Figure 5.1: System Architecture

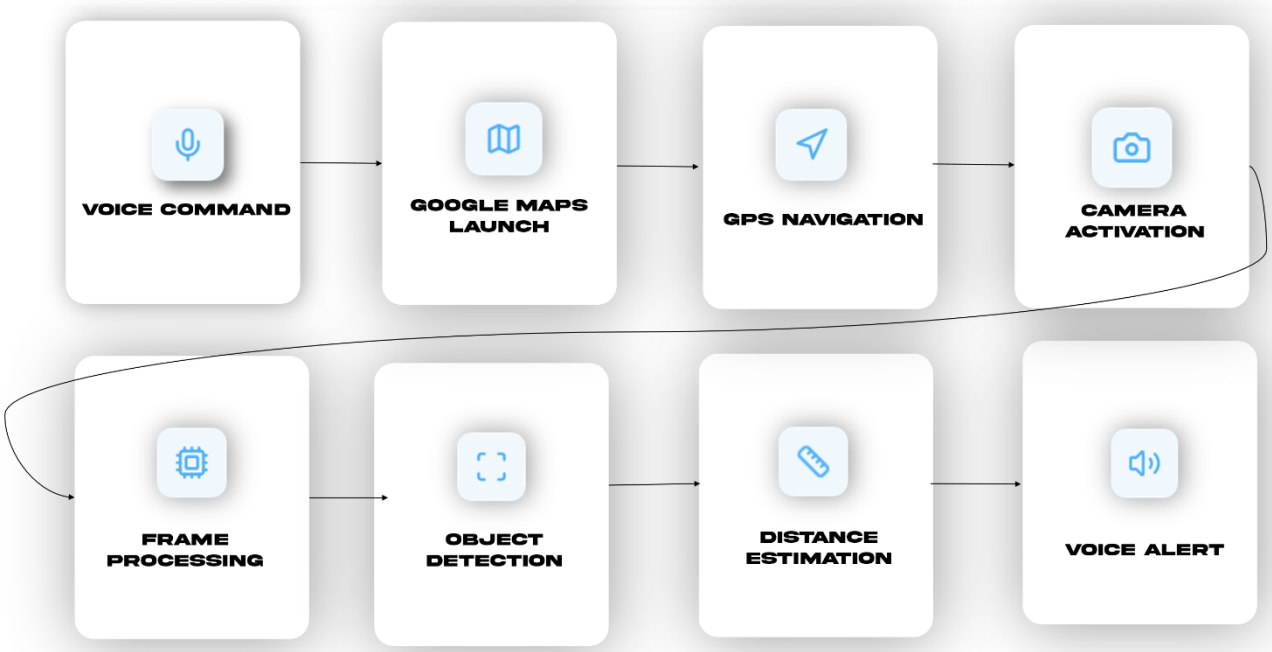


Figure 5.2: Workflow of VisionMate

5.2. UML Diagrams

Unified Modelling Language (UML) diagrams are visual representations used in software engineering to depict the design and structure of a system. UML diagrams provide a standardized way to communicate complex systems and their components, relationships and behaviours. They help software developers, designers and stakeholder understand the system architecture, design patterns and functionalities.

UML diagrams can range from high-level views, such as use case diagrams and activity diagrams, to detailed views, such as class diagrams and sequence diagrams.

- Use case diagram.
- Class diagram.
- Activity diagram.
- Sequence diagram.

5.2.1. Use Case Diagram

A Use Case diagram illustrates interactions between users and a system, showing how users interact with the system to achieve specific goals or tasks, using ovals to represent use cases and stick figures for actors.

The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted. Interaction among actors is not shown on the use case diagram. If this interaction is essential to a coherent description of the desired behavior, perhaps the system or use case boundaries should be re-examined. Alternatively, interaction among actors can be part of the assumptions used in the use case.

Use cases:

A use case describes a sequence of actions that provide something of measurable value to an actor and is drawn as a horizontal ellipse.

Actors:

An actor is a person, organization, or external system that plays a role in one or more interactions with the system.

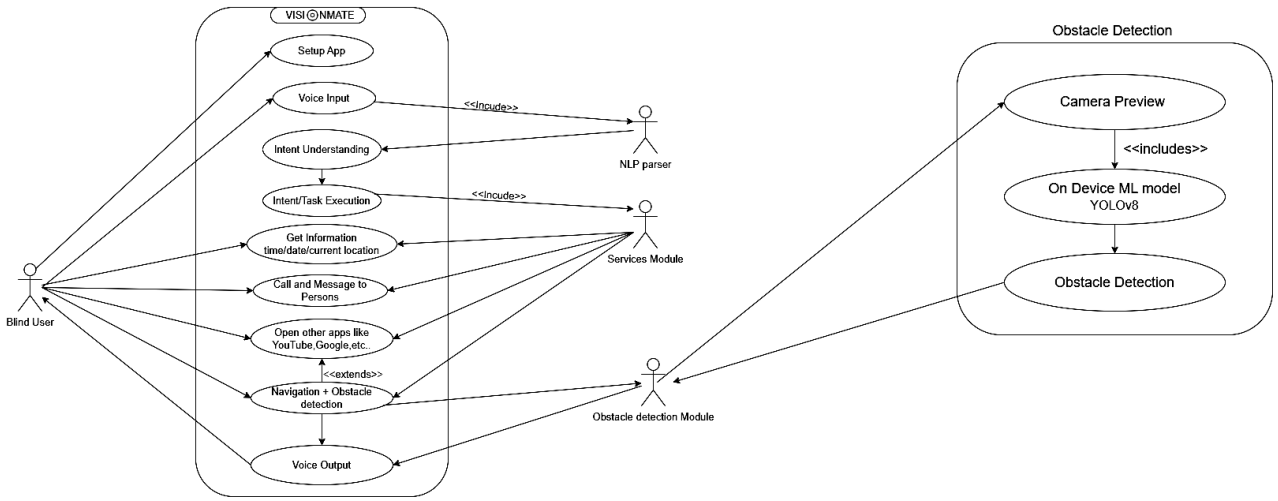


Figure 5.3: Use Case Diagram

5.2.2. Class Diagram

A class diagram illustrates the structure of a system by showing the classes, their attributes, methods, and relationships between them, helping to visualize the static design of software systems.

Class diagram describes the attributes and operations of a class and also the constraints imposed on the system. The class diagrams are widely used in the modelling of object-oriented systems because they are the only UML diagrams, which can be mapped directly with object-oriented languages.

Class diagram shows a collection of classes, interfaces, associations, collaborations, and constraints. It is also known as a structural diagram.

Purpose of Class Diagrams:

The purpose of class diagram is to model the static view of an application. Class diagrams are the only diagrams which can be directly mapped with object-oriented languages and thus widely used at the time of construction.
UML diagrams like activity diagram, sequence diagram can only give the sequence flow of the application, however class diagram is a bit different. It is the most popular UML diagram in the coder community.
The purpose of the class diagram can be summarized as − •

Analysis and design of the static view of an application.

- Describe responsibilities of a system.
- Base for component and deployment diagrams.
- Forward and reverse engineering.

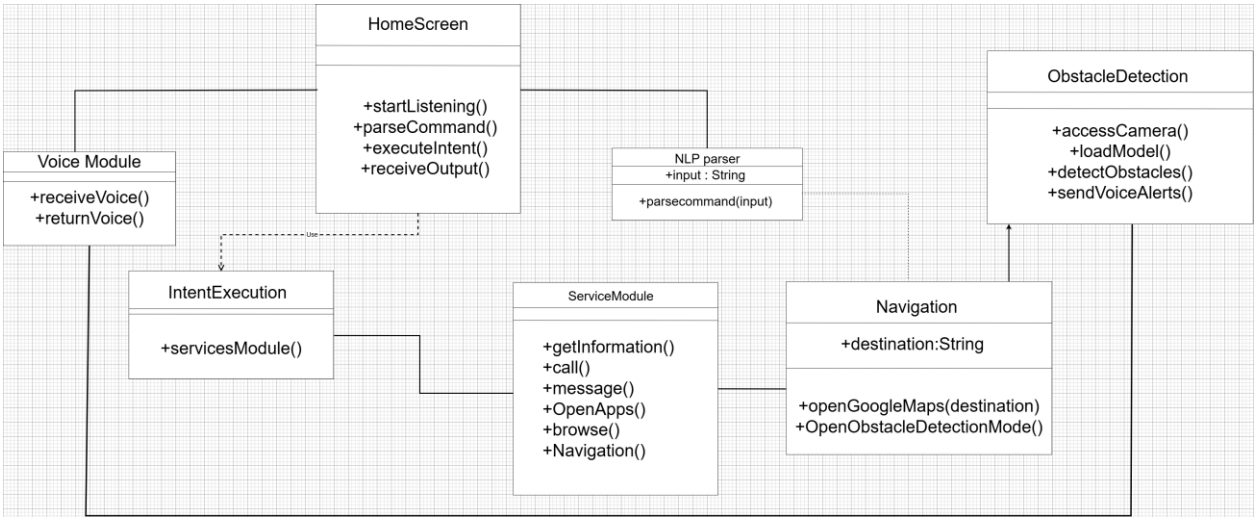


Figure 5.4: Class Diagram

5.1.1. Activity Diagram

An activity diagram depicts the flow of activities or processes within a system, illustrating the sequence of actions, decisions, and transitions from one activity to another using nodes and arrows. **When to Use Activity Diagram:**

Activity Diagrams describe how activities are coordinated to provide a service which can be at different levels of abstraction. Typically, an event needs to be achieved by some operations, particularly where the operation is intended to achieve a number of different things that require

coordination, or how the events in a single use case relate to one another, in particular, use cases where activities may overlap and require coordination. It is also suitable for modelling how a collection of use cases coordinates to represent business workflows Identify candidate use cases, through the examination of business workflows Identify pre- and post-conditions (the context) for use cases Model workflows between/within use cases Model complex workflows in operations on objects Model in detail complex activities in a high-level activity Diagram.

Activity Diagram Notation:

1. **Activity** - Is used to represent a set of actions
2. **Action** - A task to be performed
3. **Control Flow** - Shows the sequence of execution
4. **Object Flow** - Show the flow of an object from one activity to another activity.
5. **Initial Node** - Portrays the beginning of a set of actions or activities
6. **Activity Final Node** - Stop all control flows and object flows in an activity (or action)
7. **Object Node** - Represent an object that is connected to a set of Object Flows
8. **Decision Node** - Represent a test condition to ensure that the control flow or object flow only goes down one path
9. **Merge Node** - Bring back together different decision paths that were created using a decisionnode.
10. **Fork Node** - Split behaviour into a set of parallel or concurrent flows of activities (or actions)

11. **Join Node** - Bring back together a set of parallel or concurrent flows of activities (or actions).

12. **Swimlane and Partition** - A way to group activities performed by the same actor on an activity diagram or to group activities in a single thread.

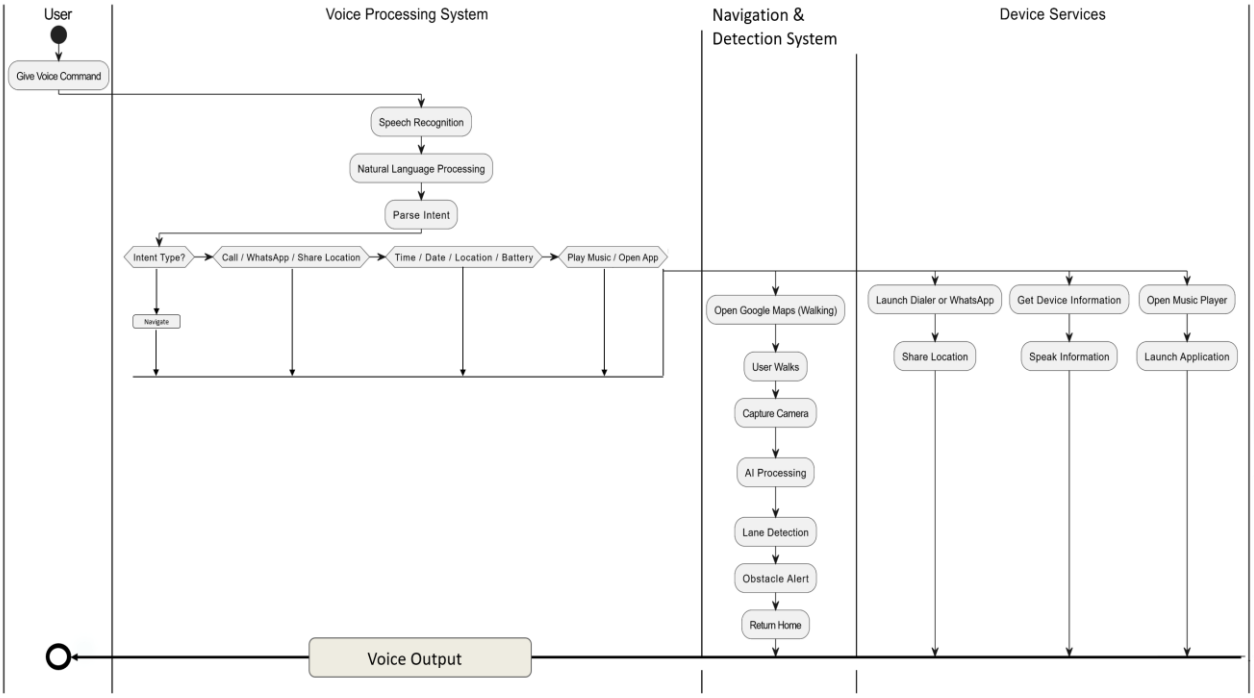


Figure 5.5: Activity Diagram

5.1.2. Sequence Diagram

A sequence diagram depicts interactions between objects or components over time, showing the order of messages exchanged to accomplish tasks or scenarios in software systems. It helps visualize dynamic behaviour and communication patterns within the system.

Sequence Diagram Notations

Actors:

An actor in a UML diagram represents a type of role where it interacts with the system and its

objects. We represent an actor in a UML diagram using a stick person notation.

Lifelines:

A lifeline is a named element which depicts an individual participant in a sequence diagram. So basically, each instance in a sequence diagram is represented by a lifeline. Lifeline elements are located at the top in a sequence diagram. We display a lifeline in a rectangle called head with its name and type.

Messages:

Communication between objects is depicted using messages. The messages appear in a sequential order on the lifeline. We represent messages using arrows. Lifelines and messages form the core of a sequence diagram.

Guards:

To model conditions, we use guards in UML. They are used when we need to restrict the flow of messages on the pretext of a condition being met.

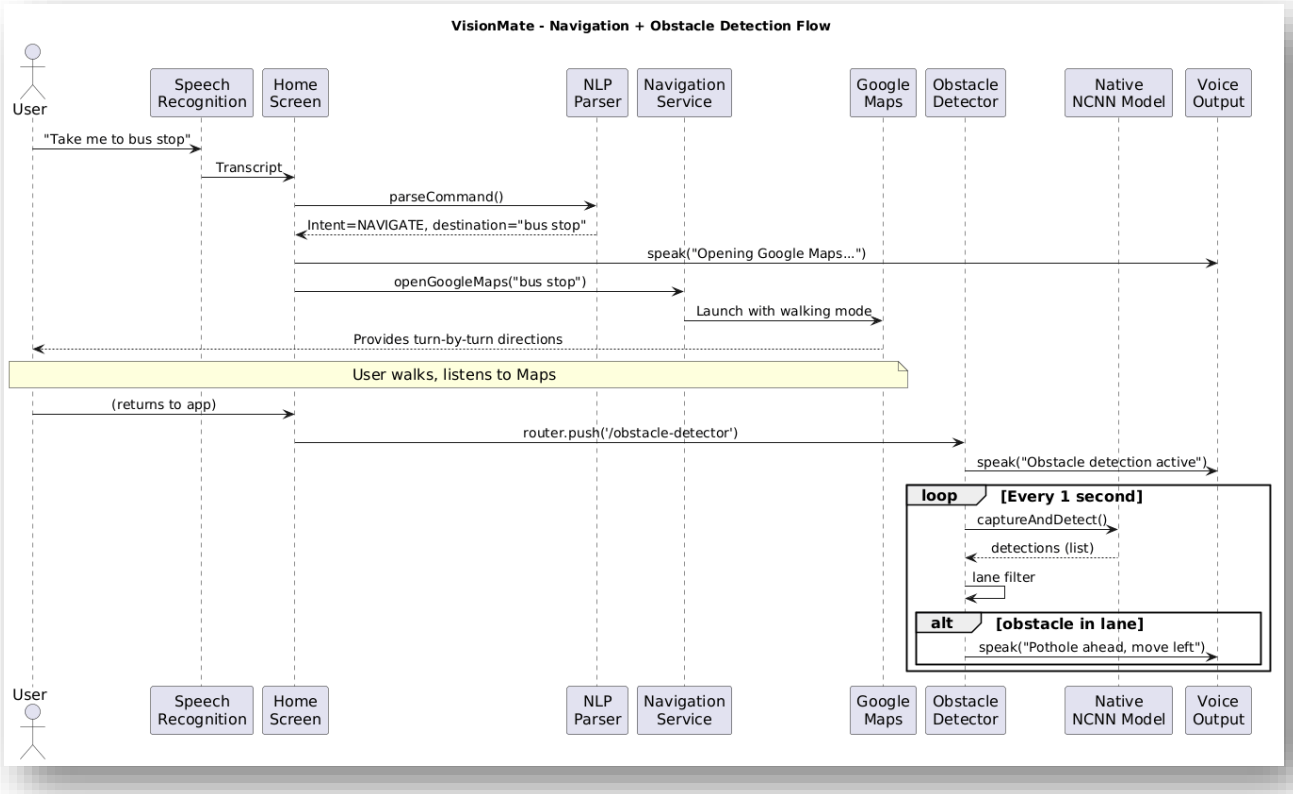


Figure 5.6: Sequence Diagram

CHAPTER - 6
TECHNOLOGY

The technology description defines the set of tools, frameworks, programming languages, and libraries used in the development of the VisionMate AI-powered assistive navigation system**.** The system is designed using modern mobile, machine learning, and native processing technologies to ensure real-time performance and accessibility.

6.1. Python

Guido van Rossum is the author of Python, a high-level object-oriented programming language. It is also known as a general-purpose programming language because it is employed in practically every industry we can imagine, as shown in the examples below:

• Web Development

• Software Development

• Game Development

• AI & ML

• Data Analytics

Python's versatility is evident in its wide range of applications across various domains. In web development, frameworks like Django and Flask empower developers to build scalable and efficient web applications. In data science and analytics, libraries such as Pandas, NumPy, and Matplotlib enable researchers and analysts to analyse and visualize data effectively. Python in machine learning and artificial intelligence is demonstrated by libraries like TensorFlow, PyTorch and Scikit-learn, which provide powerful tools for developing and deploying machine learning models.

6.1.1. Features of python

1. Simple and easy to learn
2. Freeware and Open Source
3. High Level Programming language
4. Platform Independent
5. Portability
6. Dynamically typed
7. Both Procedure Oriented and Object Oriented
8. Interpreted

6.2. Python Libraries

A python library is a collection of related modules. It contains bundles of code that can be used repeatedly in different programs. It makes python programming simpler and convenient for the programmer. Python libraries play a very vital role in fields of Machine Learning, Data science.

6.2.1 PyTorch

PyTorch is an open-source deep learning framework used for building and training neural networks. It provides flexibility and efficient computation, especially for computer vision and AI applications.

Features of PyTorch:

- Dynamic computation graph for flexible model building
- Supports GPU acceleration for faster training
- Widely used for deep learning and computer vision
- Easy integration with Python
- Strong support for research and production models

6.3. React Native

React Native is an open-source framework used for developing cross-platform mobile applications using JavaScript and TypeScript. It allows developers to build mobile apps for both Android and iOS using a single codebase, while still providing native-like performance.

Features of React Native:

- Enables cross-platform mobile application development
- Uses reusable UI components for faster development
- Provides near-native performance for mobile apps
- Strong community support and third-party libraries
- Supports integration with native Android and iOS modules

6.4 Expo Framework (Expo Go & EAS Build Platform)

Expo is a development framework built on top of React Native that simplifies mobile app development, testing, and deployment. It provides ready-to-use tools and services for handling device features and building mobile applications efficiently.

Features of Expo:

- Expo Go: Allows instant testing of apps on real devices without native setup
- EAS Build: Cloud-based service for generating production-ready APK and iOS builds
- Provides easy access to device features like camera, audio, and sensors
- Reduces complexity of native Android/iOS configuration
- Supports faster development and deployment cycle

6.5. TypeScript

TypeScript is a strongly typed superset of JavaScript that improves code quality by adding static typing and object-oriented features. It helps in building large-scale applications with better structure and fewer runtime errors.

Features of TypeScript:

- Provides static typing for better code safety
- Improves code readability and maintainability
- Reduces runtime errors during development
- Supports modern JavaScript features
- Widely used in large-scale mobile and web applications

6.6. YOLOv8 (You Only Look Once)

YOLOv8 is a deep learning-based object detection algorithm designed for real-time image and video processing. It is capable of detecting multiple objects in a single frame with high speed and accuracy.

Features of YOLOv8:

- Real-time object detection with high accuracy
- Supports detection of multiple objects simultaneously
- Optimized for speed and efficiency
- Nano version is suitable for mobile and edge devices
- Widely used in computer vision applications

6.7 NCNN Framework

NCNN is a high-performance neural network inference framework optimized for mobile and embedded devices. It is designed to run deep learning models efficiently without requiring cloud processing.

Features of NCNN:

- Lightweight and optimized for mobile devices
- Provides fast real-time inference
- Works without internet (offline processing)
- Low memory and CPU usage
- Suitable for edge AI applications

6.8 C++ and JNI (Java Native Interface)

C++ is a high-performance programming language used for system-level development and performance-critical applications. JNI is used to connect Android applications with native C++ code.

Features:

- High-speed execution for real-time processing
- Used for performance-critical AI inference tasks
- JNI enables communication between Android and native code
- Supports integration of machine learning models
- Improves overall system efficiency

6.9 Kotlin

Kotlin is a modern programming language officially supported for Android development. It is fully interoperable with Java and provides improved safety and readability.

Features of Kotlin:

- Modern and concise syntax
- Fully supported for Android development
- Interoperable with Java
- Reduces boilerplate code
- Improves app stability and performance

6.10. CMake

CMake is a build system used to manage the compilation of C++ code across different platforms. It is widely used in Android native development.

Features of CMake:

- Cross-platform build system
- Used for compiling native C++ code
- Automates build configuration process
- Supports large-scale projects
- Integrates with Android NDK development

6.11 Integrated Development Environment

Integrated Development Environment, combines tools like code editor, debugger, and compiler to streamline software development, enhancing productivity and efficiency for programmers.

6.11.1 Visual Studio Code

Visual Studio code, often abbreviated as VS Code, is a widely used source code editor developed by Microsoft. It supports various programming languages and provides features like debugging, syntax highlighting, and version control integration.

Features of vs code

1. Intelligent Code Editing: VS Code offers advanced code editing capabilities such as syntax highlighting, auto-completion, code refactoring and snippet support to enhance developer productivity.
2. Integrated Terminal: It comes with an integrated terminal that allows developers to execute commands, run scripts, and perform various tasks without leaving the editor.
3. Debugging Support: VS code provides built-in debugging support for multiple programming languages, enabling developers to debug their applications directly from the editor.

6.12 Kaggle Notebook

Kaggle Notebook is a cloud-based development environment provided by Kaggle that allows developers and researchers to write, execute, and share machine learning and data science code directly in the browser. It provides free access to GPUs, datasets, and pre-installed machine learning libraries, making it an ideal platform for training and testing AI models.

In the VisionMate project, Kaggle Notebook was used for dataset analysis, preprocessing, and training the YOLOv8 object detection model before deploying it into the mobile application.

Features of Kaggle Notebook:

- Cloud-based coding environment accessible from any device
- Provides free GPU and TPU support for training deep learning models
- Pre-installed libraries such as PyTorch, NumPy, Pandas, and OpenCV
- Easy access to thousands of public datasets for machine learning
- Allows sharing, collaboration, and version control of experiments
- Enables direct dataset import from Kaggle without manual downloads

6.13 Introduction to Deep Learning:

Deep learning is an advanced branch of machine learning that uses artificial neural networks to learn patterns directly from data. Unlike traditional machine learning methods that require manual feature extraction, deep learning models automatically identify important features from large datasets.

Deep learning has gained significant importance in areas such as computer vision, speech recognition, and natural language processing. Its effectiveness is mainly due to the availability of large datasets, powerful computational resources like GPUs, and improved neural network architectures.

In computer vision applications, deep learning models are widely used for tasks such as image classification and object detection. Algorithms like **YOLO (You Only Look Once)** enable real-time detection of multiple objects in images or video streams with high accuracy.

In this project, deep learning techniques are used to develop **VisionMate**, an assistive navigation system that detects road obstacles using the **YOLOv8 Nano object detection model**. The system processes camera input, identifies obstacles such as potholes and speed bumps, and provides voice alerts to help visually impaired users navigate safely.

CHAPTER - 7

DATASET DESCRIPTION

7.1. Dataset

Dataset Name: RDD2022 Road Damage Dataset

The dataset used for road damage detection in this project is sourced from the publicly available RDD2022 Road Damage Dataset available on Kaggle**.** It is a large-scale dataset containing annotated road images captured using smartphone cameras in different countries. The dataset is widely used in research for detecting road surface damages such as potholes and cracks.

• **Source:** https://www.kaggle.com/datasets/aliabdelmenam/rdd-2022

• **Type:** RGB (color) road images

• **Image Resolution:** Various resolutions (typically around 720×720 to 1280×720 pixels)

• **Format:** JPG

The dataset contains road images collected from six different countries and annotated with bounding boxes representing different types of road damages.

Road Damage Type	Class ID
Longitudinal Crack	**0**
Transverse Crack	**1**
Alligator Crack	**2**
Pothole	**3**

Table 7.1: Total Number of class by Category

Country	Number of Images
Japan	16,000
India	9,053
Czech Republic	3,137
Norway	7,067
United States	5,533
China	6,630
Total	**47,420**

Table 7.2: Total Number of Images by Country in RDD2022 Dataset

Figure 7.1: Potholes images from RDD2022 dataset

Dataset-2

Dataset Name: Speed Bump Dataset

In addition to the road damage dataset, a speed bump dataset was used to train the model to recognize speed bumps on roads. The dataset contains images of road surfaces categorized into different road conditions including bumps, potholes, cracks, and normal road surfaces.

• **Source:** https://www.kaggle.com/datasets/ziya07/speed-bump-dataset

• **Type:** RGB (color) road images

• **Image Resolution:** Resized to **640×640 pixels** during preprocessing for training

• **Format:** JPG / PNG

The dataset is organized into four categories representing different road surface conditions.

Category	Class ID
Road	**0**
Crack	**1**
Pothole	**2**
Bump	**3**

Table 7.3: Total Number of class by Category

Road Condition	Number of Images
Road	1200
Crack	300
Pothole	300
Bump	300
Total	**2100**

Table 7.4: Total Number of Images by Category

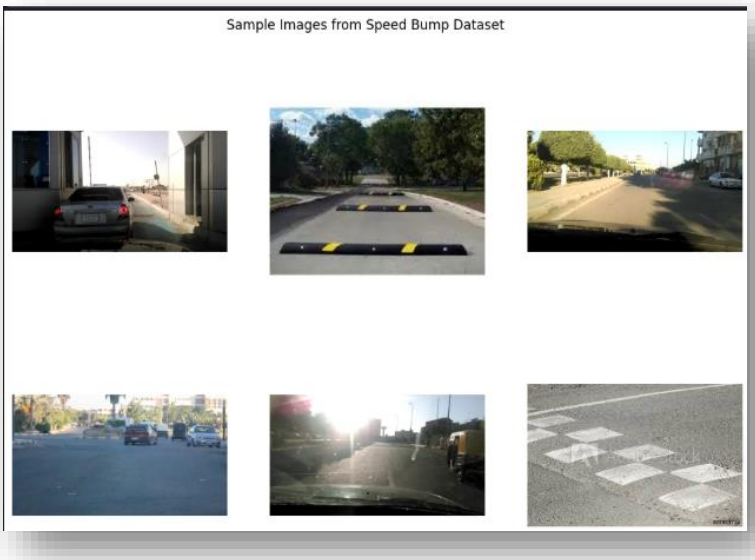


Figure 7.2: SpeedBumps images from the dataset

CHAPTER - 8

ALGORITHMS

8.1. YOLOv8 Object Detection Algorithm

YOLO (You Only Look Once) is a deep learning based object detection algorithm designed for real-time detection of objects in images and video streams. Unlike traditional object detection methods that process images in multiple stages, YOLO analyzes the entire image in a single pass and predicts bounding boxes and object classes simultaneously.

YOLOv8 is the latest version of the YOLO family and provides improved accuracy, faster detection speed, and better model efficiency. The **YOLOv8 Nano (YOLOv8n)** model is a lightweight version specifically designed for real-time applications and edge devices.

In this project, YOLOv8n is used to detect **road obstacles** such as potholes, cracks, and speed bumps from camera images. The model processes the input image, identifies objects, and generates bounding boxes around detected obstacles. These detections are then converted into voice alerts to assist visually impaired users during navigation.

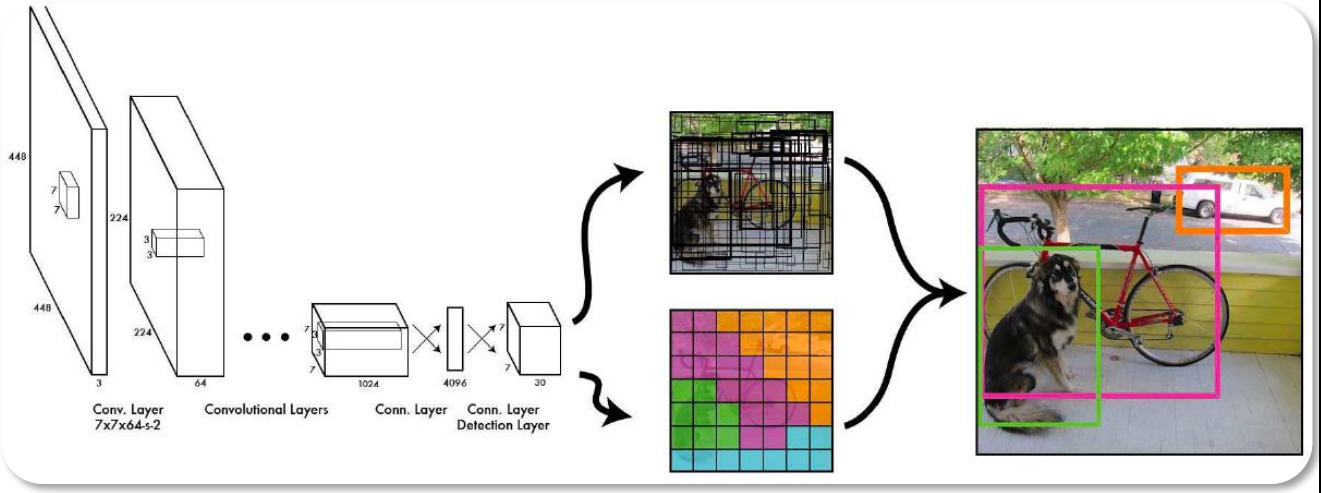


Fig-8.1 : How it use bounding boxes

8.1.1 Convolution

In this operation, a small tensor is applied across an input image which is called as a kernel or a filter. Its main task is to extract the important features from the image. After this operation a feature map is generated which is computed through element wise multiplication of each value in image and kernel. This feature map will contain the vertical and horizontal edges which are enhanced in next step.

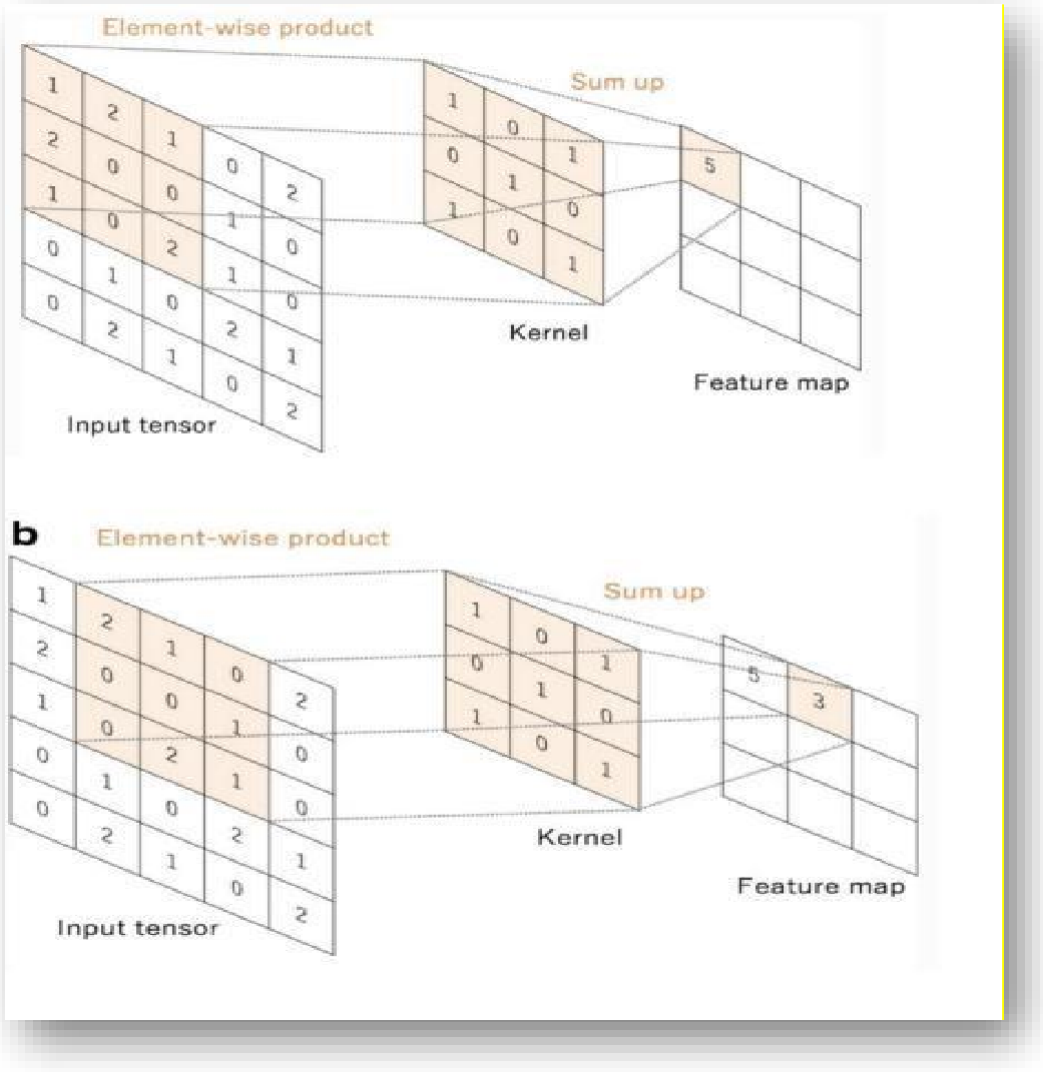


Figure 8.2: Showing convolution operation

8.1.2 Activation Function

Activation functions are used in neural networks to introduce non-linearity, allowing the model to learn complex patterns from input data. These functions transform the output of neurons before passing it to the next layer.

A commonly used activation function is **ReLU (Rectified Linear Unit)**:

f(x)=max⁡(0,x)f(x) = \max(0, x)f(x)=max(0,x)

Activation functions help deep learning models such as YOLO detect important features from images.

8.1.3.Pooling Layer

YOLO models internally use **feature extraction networks**, which include pooling or similar down-sampling operations.

Pooling Layer

Pooling layers are used to reduce the spatial size of feature maps while retaining the most important information. This helps reduce computational complexity and improves model efficiency. Pooling operations allow deep learning models to focus on the most relevant features in an image.

Max Pooling

Max pooling is a common pooling technique where the maximum value from a small region of the feature map is selected. This helps highlight the most prominent features detected by the network.

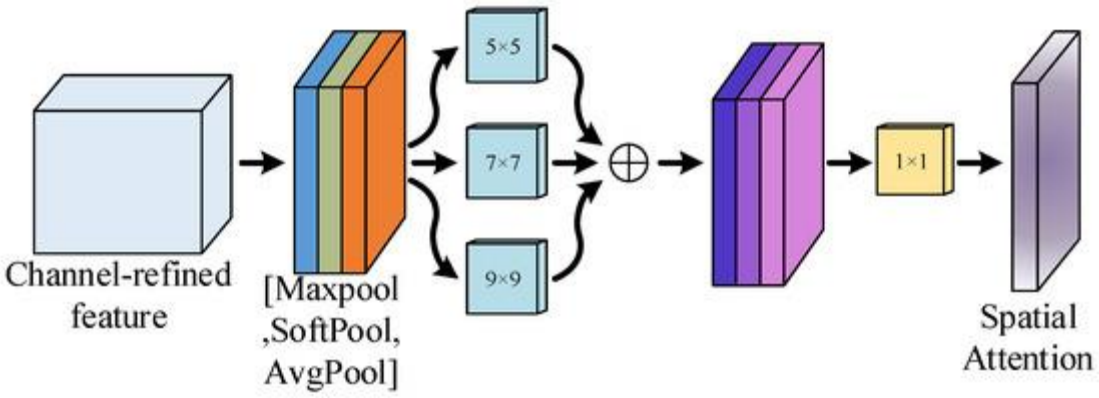


Figure-8.3 MaxPooling

8.1.4 Fully Connected Layer

Fully connected layers combine features extracted by convolution layers and perform prediction tasks. In object detection models like YOLO, specialized detection heads are used instead of traditional fully connected classification layers.

8.1.5 Loss Function

Loss functions are used during training to measure the difference between predicted outputs and actual labels. The objective of training is to minimize this loss value.

In object detection models like YOLOv8, the loss function combines multiple components such as:

- Bounding box regression loss
- Object classification loss
- Objectness loss

These losses help the model accurately detect and localize objects in an image.

8.1.6 Optimizer

Optimizers are algorithms used to update the weights of a neural network in order to minimize the loss function during training. In this project, the YOLOv8 model uses modern optimization techniques to improve training efficiency and model accuracy.

One commonly used optimizer is **Adam**, which combines momentum and adaptive learning rate methods to achieve faster convergence.

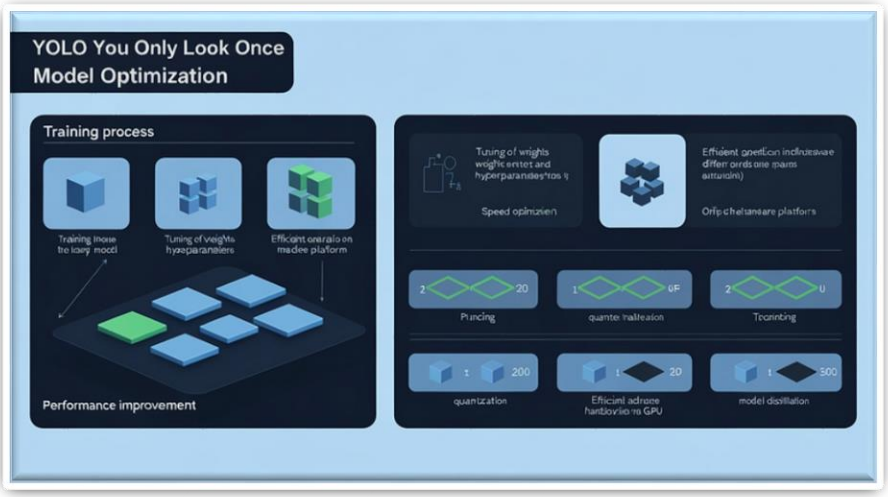


Figure 8.4 – Model Optimization

8.2 Evaluation Metrics

The performance of the trained YOLOv8 model is evaluated using the following metrics:

Precision

Precision measures the proportion of correctly detected objects among all detected objects.

Precision = TP / (TP + FP)

Recall

Recall measures the ability of the model to detect all relevant objects in the dataset.

Recall = TP / (TP + FN)

Mean Average Precision (mAP)

Mean Average Precision (mAP) is the most important evaluation metric in object detection tasks. It measures the overall accuracy of the model in detecting and localizing objects.

Higher mAP values indicate better model performance.

CHAPTER - 9

SYSTEM IMPLIMENTATION

9.1 Dataset loading and preprocessing for RDD2022

9.1.1 Importing all the dependencies

```
import os
import random
import matplotlib.pyplot as plt
from ultralytics import YOLO
from PIL import Image
```

9.1.2 Setting the structure of the dataset

```
[4]: dataset_path = "/kaggle/input/datasets/aliabdelmenam/rdd-2022"
     print("Dataset folders:")
     print(os.listdir(dataset_path))

Dataset folders:
['RDD_SPLIT']
```

```
[8]: train_images = os.listdir(dataset_path + "/RDD_SPLIT/train/images")
     val_images = os.listdir(dataset_path + "/RDD_SPLIT/val/images")
     test_images = os.listdir(dataset_path + "/RDD_SPLIT/test/images")

     print("Number of training images:", len(train_images))
     print("Number of validation images:", len(val_images))
     print("Number of testing images : " , len(test_images))

Number of training images: 26869
Number of validation images: 5758
Number of testing images :  5758
```

9.1.3 Watching some random images from the dataset

```
[1]: import os
     import random
     import matplotlib.pyplot as plt
     from PIL import Image

     # Folder path
     folder_path = "/kaggle/input/datasets/aliabdelmenam/rdd-2022/RDD_SPLIT/train/images"

     # Get all images
     images = os.listdir(folder_path)

     # Display random sample images
     plt.figure(figsize=(12,8))

     for i in range(6):

         img_path = os.path.join(folder_path, random.choice(images))
         img = Image.open(img_path)

         plt.subplot(2,3,i+1)
         plt.imshow(img)
         plt.axis("off")

     plt.suptitle("Sample Images from Speed Bump Dataset")

     plt.show()
```

Figure 9.1: Visualizing Images

9.1.4 Loading the YOLOv8n model

```
[6]: !pip install ultralytics -q
     from ultralytics import YOLO
     model = YOLO("yolov8n.pt")

     print("YOLOv8n model loaded successfully")
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 19.5 MB/s eta 0:00:00a 0:00:01
Creating new Ultralytics Settings v0.0.6 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolov8n.pt': 100% ━━━━━━━━━━━━ 6.2MB 78.9MB/s 0.1s
YOLOv8n model loaded successfully
```

9.1.5. Training the Model

```
[ ]: model.train(
         data="/kaggle/working/rdd2022.yaml",
         epochs=50,
         imgsz=640,
         batch=16,
         device=0
     )
```

.

9.1.6 Model training and epochs running

```
Image sizes 640 train, 640 val
Using 4 dataloader workers
Logging results to /kaggle/working/runs/detect/train-4
Starting training for 50 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       1/50      2.11G      1.999      3.588      1.755         32        640: 100% ━━━━━━━━━━━━ 1517/1517 3.2it/s 7:47<0.3s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 163/163 3.8it/s 43.3s0.3ss
                   all       5214       7827      0.321      0.294      0.228     0.0982

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       2/50      2.58G      1.996      2.754      1.734         31        640: 100% ━━━━━━━━━━━━ 1517/1517 3.9it/s 6:34<0.2s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 163/163 4.0it/s 40.8s0.3ss
                   all       5214       7827      0.309      0.304      0.233     0.0974

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       3/50      2.58G      2.067      2.673      1.814         36        640: 57% ━━━━━━━━━━━━ 862/1517 4.7it/s 3:37<2:208
```

9.2 Data Splitting

To train the model effectively, the dataset was divided into two subsets:

- **Training dataset**
- **Validation dataset**

The dataset was split in the ratio of **80% for training and 20% for validation**.

This splitting ensures that the model learns patterns from the training dataset while its performance is evaluated using the validation dataset.

The training set is used to update the model weights, while the validation set is used to monitor the model's performance during training and prevent overfitting.

.2.1 Preprocessing

Before training the model, several preprocessing steps were applied to the dataset.

1. **Image Resizing**
   All images were resized to **640 × 640 pixels** during training. This ensures consistency in input dimensions and improves training efficiency.

2. **Normalization**
   Pixel values were automatically normalized by the YOLOv8 framework.

3. **Data Augmentation**
   To improve model generalization and reduce overfitting, several augmentation techniques were applied during training, including:

- Random horizontal flipping
- Random scaling
- Random brightness adjustment
- Random rotation

These techniques create variations of the training images, enabling the model to learn more robust features.

9.2.2 Model Architecture

The object detection model used in this project is **YOLOv8n**, which is the lightweight version of the YOLOv8 architecture.

YOLO (You Only Look Once) is a real-time object detection algorithm that detects objects in images using a single neural network.

The YOLOv8 architecture consists of three main components:

Backbone

The backbone is responsible for extracting features from input images.

Neck

The neck aggregates features from different layers to improve object detection at multiple scales.

Head

The detection head predicts:

- bounding box coordinates
- object confidence score
- class probabilities

The **YOLOv8n model** was selected because it provides a good balance between **accuracy and computational efficiency**, making it suitable for real-time road damage detection systems.

9.2.3 Model Compilation

During training, the YOLOv8 framework automatically configures the model with appropriate optimization techniques.

The following configurations were used:

Parameter	Value
Optimizer	SGD
Learning Rate	Automatically adjusted
Batch Size	16
Image Size	640 × 640
Epochs	50

The loss function used in YOLOv8 consists of multiple components:

- **Bounding box regression loss**
- **Classification loss**
- **Objectness loss**

These losses are optimized together to improve detection accuracy.

9.3 Model Training and Evaluation

Model training is the process of enabling the neural network to learn patterns from the dataset. During training, the model adjusts its internal weights to minimize the difference between predicted outputs and actual labels.

The YOLOv8 model was trained using the training dataset for **50 epochs** with a **batch size of 16**.

The validation dataset was used to monitor the model's performance during training.

Key evaluation metrics include:

- **Precision**
- **Recall**
- **Mean Average Precision (mAP)**

These metrics help evaluate how accurately the model detects road damages in images.

9.3.1 Prediction

After training, the model was used to predict road damages on new images.

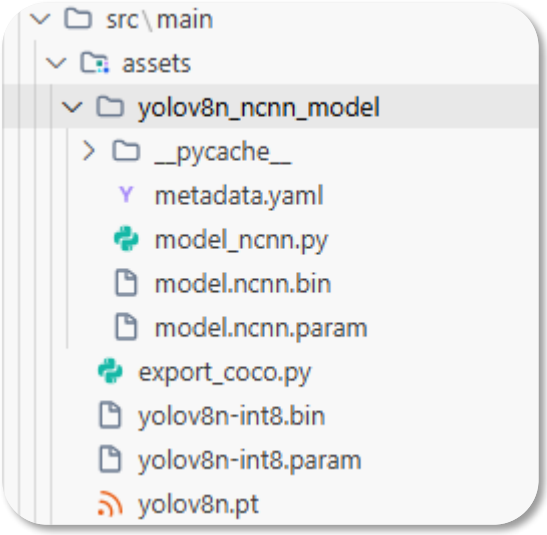
The prediction process involves the following steps:

1. Loading the trained YOLOv8 model
2. Providing an input image
3. Detecting objects present in the image
4. Drawing bounding boxes around detected road damages

The model outputs:

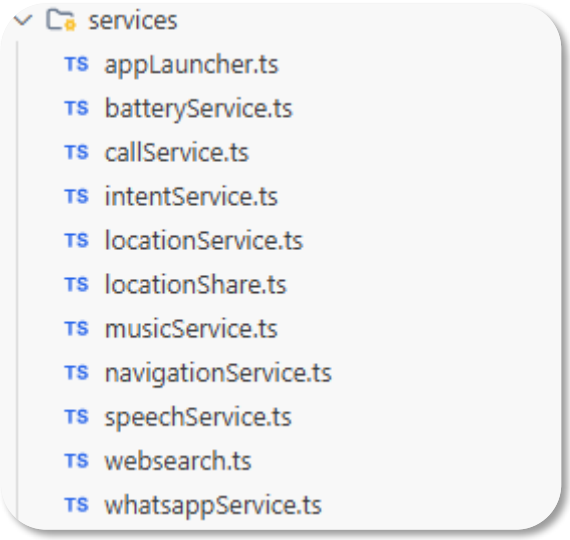
- detected class label
- confidence score
- bounding box location

9.4 Folder structure and Important Code



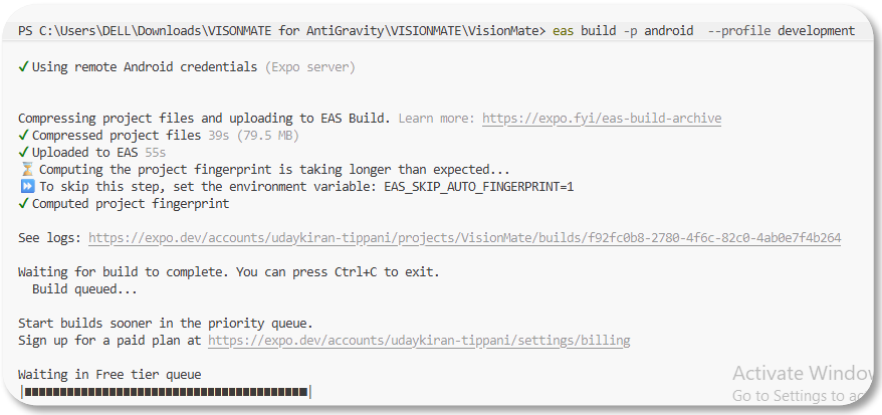
Main MI Models Folder filled with model

Services module



The main files for all the functionalities

Building the android .apk file



Running the Expo Go Metro Bundling

NavigationService .ts

```
import * as Linking from 'expo-linking';
import * as Location from 'expo-location';
import { Platform } from 'react-native';

export const openGoogleMaps = async (destination: string) => {
  try {
    const { status } = await Location.requestForegroundPermissionsAsync();
    if (status !== 'granted') {
      console.log('Location permission denied');
      return;
    }

    if (Platform.OS === 'android') {
      const url = `google.navigation:q=${encodeURIComponent(destination)}&mode=w`;
      Linking.openURL(url);
    } else {
      const location = await Location.getCurrentPositionAsync({});
      const { latitude, longitude } = location.coords;

      const url =
`https://www.google.com/maps/dir/?api=1&origin=${latitude},${longitude}&destination=${
encodeURIComponent(destination)}&travelmode=walking`;

      Linking.openURL(url);
    }

  } catch (error) {
    console.log(error);
  }
};
```

SpeechService.ts

```
import * as Speech from 'expo-speech';
export const speak = (text: string): Promise<void> => {
  return new Promise((resolve) => {
    Speech.speak(text, {
      rate: 0.9,
      pitch: 1.0,
      language: 'en-IN',
      onDone: () => resolve(),
      onError: () => resolve(),
    });
  });
};
```

obstacle-detector.tsx

```
// Lane definition (normalised coordinates 0..1)
const LANE = {
  left: 0.3,
  right: 0.7,
  top: 0.5,
  bottom: 0.9,
};

// Cooldown per obstacle type (ms)
const ALERT_COOLDOWN = 6000;

export default function ObstacleDetector() {
  const router = useRouter();
  const [detections, setDetections] = useState<any[]>([]);
  const [modelLoaded, setModelLoaded] = useState(false);
  const cameraRef = useRef<any>(null);
  const frameInterval = useRef<ReturnType<typeof setInterval> | null>(null);
  const lastAlertTime = useRef<{ [key: string]: number }>({});
  const emptyFrames = useRef(0);
  const isProcessing = useRef(false);
  const [permission, requestPermission] = useCameraPermissions();

  // Load model and request camera permission
  useEffect(() => {
    (async () => {
      // Set audio mode to allow Google Maps to speak in background
      await Audio.setAudioModeAsync({
```

```
      allowsRecordingIOS: false,
      staysActiveInBackground: true,

      playsInSilentModeIOS: true,
      shouldDuckAndroid: true,      // Lower volume of our app when Maps speaks
      playThroughEarpieceAndroid: false,
    });

    if (!permission?.granted) {
      await requestPermission();
    }
    if (permission?.granted) {
      try {
        const loaded = await NcnnDetector.loadModel();
        setModelLoaded(loaded);
        if (loaded) {
          console.log('Model loaded');
          speak('Obstacle detection model successfully loaded. Scanning environment.');
        } else {
          console.log('Model load returned false');
          speak('Critical error. The neural network failed to initialize.');


        }
      } catch (error) {
        console.error('Failed to load model:', error);
        speak('Failed to load obstacle detection model due to an exception.');
      }
    }
  })();
  return () => {
    if (frameInterval.current) clearInterval(frameInterval.current);
  };
}, [permission]);


// Capture and detect every 1 second
useEffect(() => {
  if (permission?.granted && modelLoaded) {
    frameInterval.current = setInterval(captureAndDetect, 1000);
  }
  return () => {
    if (frameInterval.current !== null) {
      clearInterval(frameInterval.current);
    }
  };
}, [permission, modelLoaded]);
```

```
//Generating alerts for some objects
const generateAlertMessage = (det: any, direction: string): string => {
   const prefix = direction === 'left' ? 'slightly left' : direction === 'right' ? 'slightly right' :
'ahead';


   if (det.classId < 100) {
     if (det.classId === 1) {
       return `Pothole ${prefix}, move ${direction === 'center' ? 'carefully' : `to the
${direction}`}`;
     }
     if (det.classId === 0) {
       return `Speed breaker ${prefix}, slow down`;
     }
     if (det.classId === 2) {
       return `Unpaved road ${prefix}, watch your step`;
     }
     return `Obstacle ${prefix}`;
   } else {
     const cocoId = det.classId - 100;
     const objectName = COCO_CLASSES[cocoId] || "object";
     return `${objectName} ${prefix}`;
   }
 };
```

CHAPTER – 10

TESTING

The purpose of testing is to exercise the different parts of the module code to detect coding errors. After this the modules are gradually integrated into subsystems, which are then integrated themselves too eventually forming the entire system. During integration of module integration testing is performed. The goal of this is to detect designing errors, while focusing the interconnection between modules. After the system was put together, system testing is performed. Here the system is tested against the system requirements to see if all requirements were met and the system performs as specified by the requirements. Finally accepting testing is performed to demonstrate to the client for the operation of the system.

For the testing to be successful, proper selection of the test case is essential. There are two different approaches for selecting test case. The software or the module to be tested is treated as a black box, and the test cases are decided based on the specifications of the system or module. For this reason, this form of testing is also called "black box testing".

The focus here is on testing the external behaviour of the system. In structural testing the test cases are decided based on the logic of the module to be tested. A common approach here is to achieve some type of coverage of the statements in the code. The two forms of testing are complementary: one tests the external behaviour, the other tests the internal structure. Testing is an extremely critical and time-consuming activity. It requires proper planning of the overall testing process. Frequently the testing process starts with the test plan. This plan identifies all testing related activities that must be performed and specifies the schedule, allocates the resources, and specifies guidelines for testing. The test plan specifies conditions that should be tested; different units to be tested, and the manner in which the module will be integrated together.

Then for different test unit, a test case specification document is produced, which lists all the different test cases, together with the expected outputs, that will be used for testing. During the testing of the unit the specified test cases are executed and the actual results are compared with the expected outputs. The final output of the testing phase is the testing report and the error report, or a set of such reports. Each test report contains a set of test cases and the result of executing the code with the test cases. The error report describes the errors encountered and the action taken to remove the error.

10.1. Testing Techniques

Testing is a process, which reveals errors in the program. It is the major quality measure employed during software development. During testing, the program is executed with a set of conditions known as test cases and the output is evaluated to determine whether the program is performing as expected. In order to make sure that the system does not have errors, the different levels of testing strategies that are applied at differing phases of software development.

Black Box Testing

In this strategy some test cases are generated as input conditions that fully execute all functional requirements for the program. This testing has been uses to find errors in the following categories:

• Incorrect or missing functions

• Interface errors

• Errors in data structure or external database access

• Performance errors

• Initialization and termination errors.

In this testing only the output is checked for correctness. The logical flow of the datails not checked.

White Box Testing

In this testing, the test cases are generated on the logic of each module by drawing flow graphs of that module and logical decisions are tested on all the cases. It has been uses to generate the test cases in the following cases:

• Guarantee that all independent paths have been executed.
• Execute all logical decisions on their true and false sides.
• Execute all loops at their boundaries and within their operational.
• Execute internal data structures to ensure their validity.
•

10.2. Testing Strategies

Unit testing:

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application

.it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results. This System consists of 3 modules. Those are Reputation module, route discovery module, audit module. Each module is taken as unit and tested. Identified errors are corrected and executable unit are obtained.

Integration testing:

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

System Testing:

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

Functional Testing:

Functional tests provide systematic demonstrations that functions tested are specified by the business and technical requirements, system documentation, an user manuals.

Functional testing is centered on the identified classes of valid input must be following items Valid Input : accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Procedures : interfacing systems or procedures must be invoked.

CHAPTER - 11

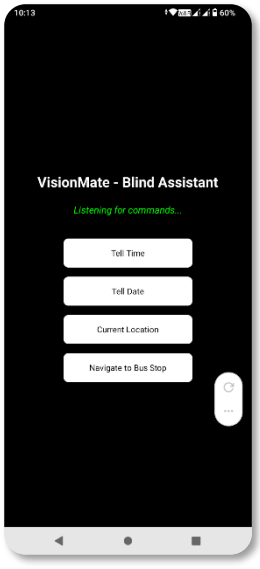
RESULTS

This section presents the results obtained from the trained YOLOv8 Nano object detection model integrated into the VisionMate mobile application. The system processes images captured from the device camera and detects road obstacles such as potholes, cracks, and speed bumps in real time. After analyzing the input image, the model generates bounding boxes around detected objects along with the corresponding class label and confidence score.

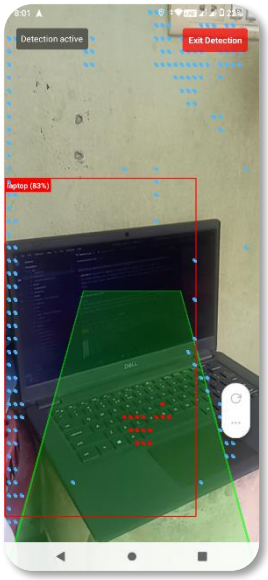
As shown in the figure, the model successfully detected road obstacles with high confidence levels. For example, potholes and road cracks were detected with confidence scores above **95%**, demonstrating the model's ability to accurately identify road damage from previously unseen images.

The detection results are visualized directly within the mobile application interface developed using **React Native and Expo**, where the detected obstacle is highlighted with a bounding box and labeled accordingly. In addition to visual output, the system also provides **voice-based alerts**, informing the user about nearby obstacles to assist visually impaired individuals during navigation.

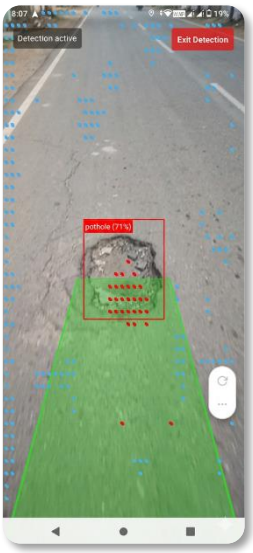
These results demonstrate that the trained YOLOv8 model can effectively detect road obstacles in real-world environments and provide timely feedback through the mobile application, validating the effectiveness of the VisionMate assistive navigation system.



User Interface



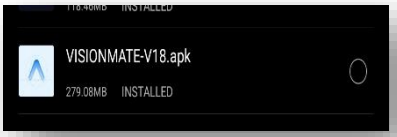
Detecting Objects

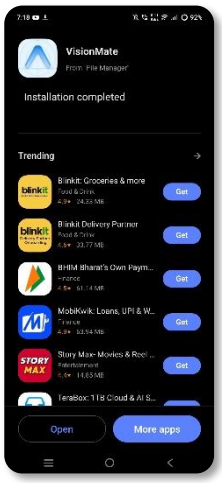


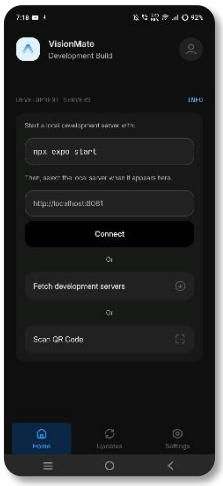
Detecting Obstacles

CHAPTER - 12
OUTPUT SCREENS

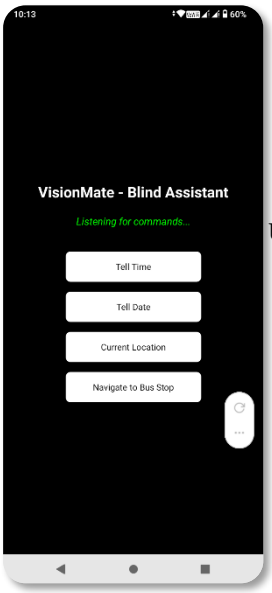
APK installation



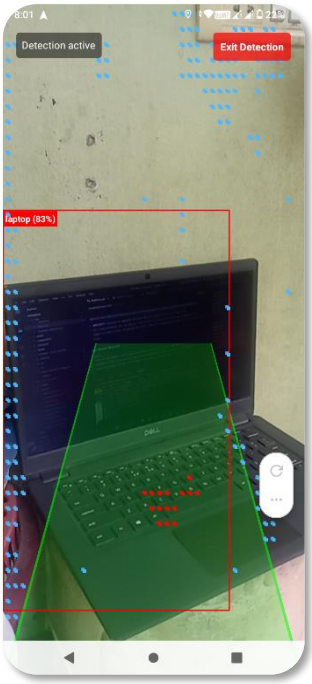




User Interface

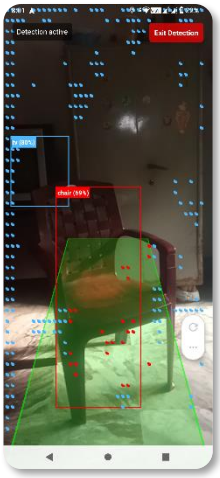


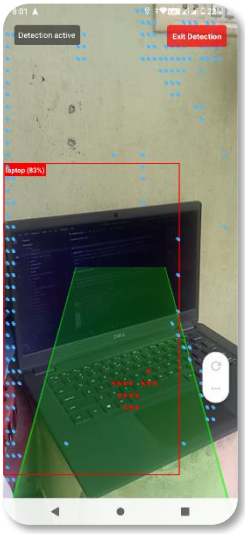
The user interface has some core functionality buttons to make it more user friendly for the blind user

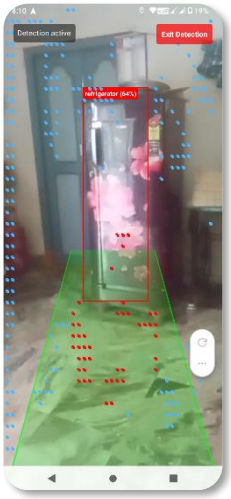


Obstacle Detection Mode interface

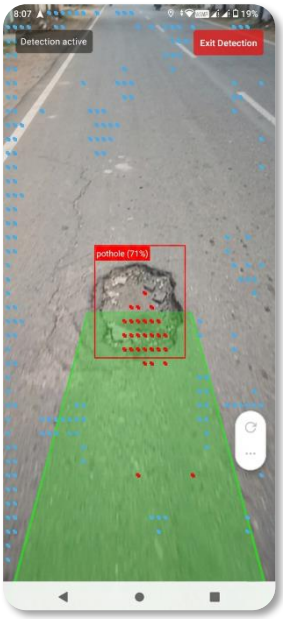
Detecting Indoor Objects





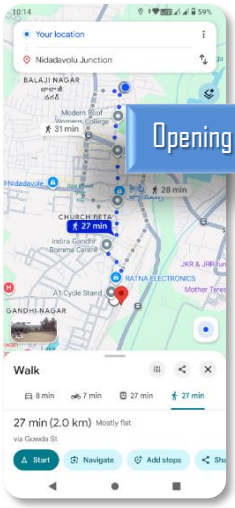


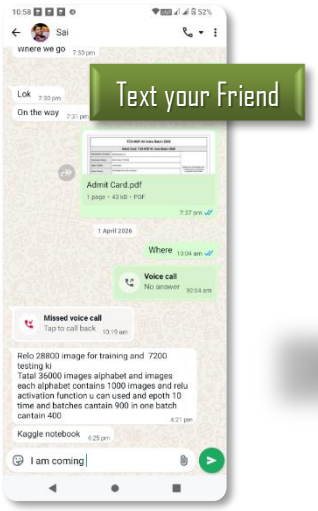
Detecting Obstacles on road



Opening Other apps

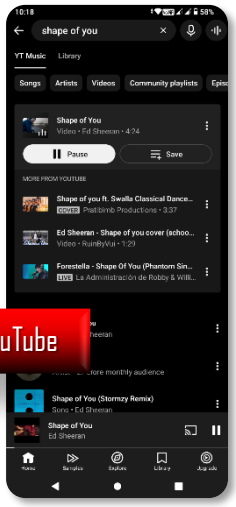
Opens the G-maps and
set destination in walking mode

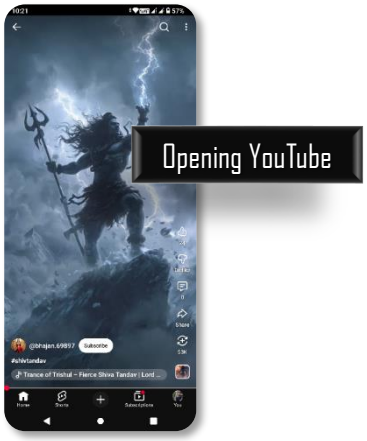




Opens Whatsapp, search for the contact
And fill the message

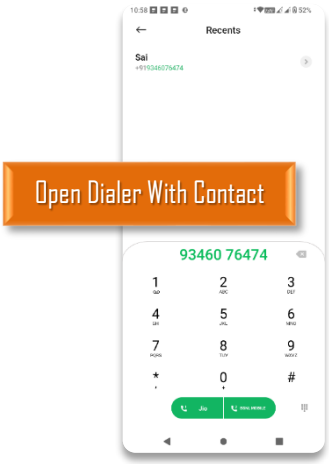
Opens the YT-Music and
search for the song





Opens YouTube and
directly plays the shorts

Opens the search engine
And search for the queries



Opens dialer and
search the contact

CHAPTER - 13

CONCLUSION

In this project, an intelligent assistive system called **VisionMate** was developed to improve mobility safety for visually impaired individuals. The system utilizes the **YOLOv8 Nano object detection model** to identify road obstacles such as potholes, cracks, and speed bumps in real time. By processing images captured from the mobile device camera, the model detects obstacles and provides timely alerts to the user.

The system integrates **navigation support and obstacle detection** within a single mobile application developed using **React Native and Expo**, making the solution portable and easy to use. By leveraging **Edge AI**, the model performs real-time detection efficiently on mobile devices without requiring heavy computational resources.

The results demonstrate that VisionMate can effectively identify road hazards and assist users in navigating safely. In addition to being **cost-effective and accessible**, the system provides a scalable foundation for future improvements such as **depth sensing, advanced navigation assistance, and smart analytics**.

Overall, VisionMate represents an important step toward the development of **intelligent assistive technologies**, contributing to safer and more independent mobility for visually impaired individuals.

CHAPTER - 14
FUTURE SCOPE

The current implementation of the VisionMate system uses the YOLOv8 Nano model to detect road obstacles such as potholes, cracks, and speed bumps, providing assistance to visually impaired users through a mobile application. Although the system demonstrates promising performance, there is significant scope for further improvement and expansion. Future enhancements may include:

1. **Real-World Dataset Expansion:**

   The current model is trained mainly on datasets such as RDD2022. Future work can include collecting more real-world road images from different cities, lighting conditions, and weather environments to improve the robustness and generalization of the detection model.

2. **Advanced Mobile Application Features:**
   The mobile application can be enhanced with additional features such as real-time navigation assistance, route safety analysis, and integration with map services to guide users through safer paths.

3. **Enhanced Voice Assistance:**
   Future versions of the system can include more advanced voice guidance and customizable audio alerts to provide better navigation support for visually impaired users.

4. **Multi-Obstacle Detection and Tracking:**
   The system can be extended to detect and track multiple obstacles simultaneously in real time, allowing users to receive continuous updates about their surroundings while walking or traveling.

5. **Integration with Wearable Devices:**
   The system could be integrated with wearable devices such as smart glasses or assistive head-mounted cameras, enabling hands-free obstacle detection and navigation support.

REFERENCES

[1] K. Naveen Kumar, Sanjeevani Sharma, Ilin Mariam Abraham, "*Blind Assistance System Using Machine Learning,"* Hindustan Institute of Technology & Science, 2022.

[2] Dr. V. Sivakumar, Neha N, Ria Sathya, Skandana G, Yashaswini R, R. Swathi, "*Smart Assistance for Visually Impaired and Blind People,*" Dayananda Sagar Academy of Technology and Management & Sree Abiraami College for Women, Thiruvalluvar University, 2022.

[3] Joseph Redmon, Ali Farhadi, *"You Only Look Once: Unified, Real-Time Object Detection*," 2016.

[4] Glenn Jocher et al., "*Ultralytics YOLOv8: Real-Time Object Detection Model*," 2023.

[5] Hiroya Maeda, Yoshihisa Sekimoto, Tatsuya Seto, "*Road Damage Detection and Classification Using Deep Neural Networks*," 2018.

[6] RDD2022 Dataset Authors, "*RDD2022: A Multi-National Road Damage Detection Dataset*," 2022.

[7] React Native Documentation, https://reactnative.dev

[8] Expo Go Documentation, https://expo.dev

[9] Kaggle Dataset – Speed Bump Detection Dataset, https://www.kaggle.com

[10] Python Official Documentation, https://www.python.org

[11] Research articles and journals related to AI-based assistive navigation systems for visually impaired individuals.